Install Flutter SDK

1. VS Code downloads **Flutter** extension.

2. Go to **View > Command Palette**.

3. Type **flutter**.

4. Select **Flutter: New Project**.

5. Select **Download SDK**.

6. Choose folder to install Flutter (the folder that doesn't require Administrator rights to avoid this error: *Failed to clone Flutter: fatal: could not create work tree dir 'flutter': Permission denied*).

7. Click **Clone Flutter**.

8. Search for **"Edit the system environment variables"** in your Start menu.

9. Click **Environment Variables**.

10. Under **User variables**, find the one named **Path** and click **Edit**.

11. Click **New** and paste the full path to the bin folder inside your flutter directory (e.g., C:\src\flutter\bin).

12. Click **OK** on all windows and **restart your terminal/command prompt/VS Code**.

13. At the terminal, run the command **flutter --version**. If version is shown, Flutter is installed successfully.

Flutter to Access Location

1. Complete steps 1 to 13 first.

2. Change the terminal directory to where this file **pubspec.yaml** is located.

3. Run the command **flutter pub add geolocator**.

Add Permissions to AndroidManifest.xml (for Android) / Info.plist (for IOS)

1. In VS Code, click **View > Command Palette**.

2. Select **Application**.

3. Pick a folder outside the Flutter SDK folder (but within the same folder with every other folder related).

4. Give it any name (like my_geo_app).

5. In VS Code, select the folder created just now.

6. For Android, click **android > app > src > main > AndroidManifest.xml**.

7. Add these two lines right after the line *<manifest xmlns:android="http://schemas.android.com/apk/res/android">*.

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
```

8. Replace the code in **lib > main.dart** with

```
import 'dart:convert';
import 'dart:io';
import 'package:flutter/material.dart';
import 'package:geolocator/geolocator.dart';
import 'package:camera/camera.dart';
import 'package:http/http.dart' as http;

Future<void> main() async {
  // Ensure that plugin services are initialized so that
`availableCameras()` can be called.
  WidgetsFlutterBinding.ensureInitialized();
  final cameras = await availableCameras();
  runApp(MaterialApp(home: DisasterAppHome(cameras: cameras)));
}

class DisasterAppHome extends StatefulWidget {
  final List<CameraDescription> cameras;
  DisasterAppHome({required this.cameras});

  @override
  _DisasterAppHomeState createState() => _DisasterAppHomeState();
}

class _DisasterAppHomeState extends State<DisasterAppHome> {
  late CameraController _controller;
  late Future<void> _initializeControllerFuture;
  String _statusMessage = "Ready to take a picture and get location.";
  String? _capturedImagePath; // Store captured image path
  bool _hasResult = false; // Track if we have a result from Gemini

  @override
  void initState() {
    super.initState();
    // To display the current output from the Camera, create a
CameraController.
    _controller = CameraController(widget.cameras.first,
ResolutionPreset.medium);
    // Initialize the controller. This returns a Future.
    _initializeControllerFuture = _controller.initialize();
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
```

```
  }

  Future<void> _processDisasterAlert() async {
    setState(() => _statusMessage = "Processing...");
    Position? position;
    XFile? image;

    try {
      // 1. Get GPS Location
      setState(() => _statusMessage = "Getting location...");
      bool serviceEnabled = await Geolocator.isLocationServiceEnabled();
      if (!serviceEnabled) throw Exception("GPS is disabled.");

      LocationPermission permission = await Geolocator.checkPermission();
      if (permission == LocationPermission.denied) {
        permission = await Geolocator.requestPermission();
        if (permission == LocationPermission.denied) throw
Exception("Location permission denied.");
      }
      position = await Geolocator.getCurrentPosition();

      // 2. Take Picture
      setState(() => _statusMessage = "Taking picture...");
      await _initializeControllerFuture;
      image = await _controller.takePicture();

      // Show the captured image
      setState(() => _capturedImagePath = image!.path);

    } catch (e) {
      setState(() => _statusMessage = "Error: $e");
      return; // Stop processing if either GPS or camera fails
    }

    // 3. Send to Gemini Backend
    setState(() => _statusMessage = "Analyzing with Gemini...");
    await _sendToBackend(position, image);
  }

  Future<void> _sendToBackend(Position position, XFile image) async {
    // IMPORTANT: Replace with your laptop's actual IP address!
    // Do not use 'localhost' if testing on a real phone.
    final url =
Uri.parse('http://YOUR_LAPTOP_IP_ADDRESS:3000/evaluate');

    try {
      // Convert image to Base64
      final bytes = await image.readAsBytes();
      String base64Image = base64Encode(bytes);

      final response = await http.post(
        url,
        headers: {"Content-Type": "application/json"},
        body: jsonEncode({
          "latitude": position.latitude,
          "longitude": position.longitude,
          "imageBase64": base64Image,
          "mimeType": "image/jpeg"
        }),
```

```
      );

      if (response.statusCode == 200) {
        final data = jsonDecode(response.body);
        setState(() {
          _statusMessage = """
AI Result: ${data['result']}
Your Location: ${position.latitude}, ${position.longitude}
""";
          _hasResult = true; // Mark that we have a result
        });
      } else {
        setState(() => _statusMessage = "Backend Error:
${response.statusCode}");
      }
    } catch (e) {
      setState(() => _statusMessage = "Error sending data: $e");
    }
  }

  void _resetToCamera() {
    setState(() {
      _capturedImagePath = null;
      _hasResult = false;
      _statusMessage = "Ready to take a picture and get location.";
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text("Disaster App")),
      body: SingleChildScrollView( // Add scrolling just in case
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.center, // Always center
horizontally
          children: [
            // Fix: Constrain the camera height so it doesn't push text
off-screen
            SizedBox(
              width: double.infinity, // Take full width to ensure
centering
              height: MediaQuery.of(context).size.height * 0.5, // Uses 50%
of screen height
              child: _capturedImagePath != null
                  ? Center(
                      child: Image.file(
                        File(_capturedImagePath!),
                        fit: BoxFit.contain,
                      ),
                    )
                  : FutureBuilder<void>(
                      future: _initializeControllerFuture,
                      builder: (context, snapshot) {
                        if (snapshot.connectionState ==
ConnectionState.done) {
                          return Center(child: CameraPreview(_controller));
                        } else {
```

```
                          return const Center(child:
CircularProgressIndicator());
                        }
                      },
                    ),
            ),
            const SizedBox(height: 20),
            Padding(
              padding: const EdgeInsets.symmetric(horizontal: 16.0),
              child: Container(
                padding: const EdgeInsets.all(12),
                decoration: BoxDecoration(
                  color: Colors.grey[200],
                  borderRadius: BorderRadius.circular(8),
                ),
                child: Text(
                  _statusMessage,
                  textAlign: TextAlign.center,
                  style: const TextStyle(fontSize: 14, fontWeight:
FontWeight.bold)
                ),
              ),
            ),
            // Show "Take New Photo" button when we have a result
            if (_hasResult) ...[
              const SizedBox(height: 16),
              ElevatedButton.icon(
                onPressed: _resetToCamera,
                icon: const Icon(Icons.camera_alt),
                label: const Text("Take New Photo"),
                style: ElevatedButton.styleFrom(
                  padding: const EdgeInsets.symmetric(horizontal: 24,
vertical: 12),
                ),
              ),
              const SizedBox(height: 80), // Space for FAB
            ],
          ],
        ),
      ),
      floatingActionButtonLocation:
FloatingActionButtonLocation.centerFloat,
      floatingActionButton: _hasResult
          ? null // Hide FAB when showing result
          : FloatingActionButton.extended(
              onPressed: _processDisasterAlert,
              icon: const Icon(Icons.emergency_share),
              label: const Text("Analyze Situation"),
            ),
    );
  }
}
```

Check Location with Android Phone

Method 1: Wireless Debugging (For Coding/Testing)

1. On Android phone,

a) Go **Settings > About phone > Software information >** Click 7 times on **Build number > OK**.

b) Back to **Settings > Developer options > On > Wireless debugging > On > Allow > Wireless debugging > Pair device with pairing code**.

2. On laptop terminal,

a) Change directory to where command **flutter pub add geolocator** is typed (refer **Flutter to Access Location** part).

b) Enter the command **adb pair [IP_ADDRESS]:[PORT]**, in which [IP_ADDRESS]:[PORT] can get achieved from the phone **Settings > Wireless debugging > Pair device with pairing code**.

c) **adb** not found? Enter the command **adb --version** at terminal. Still not found? At Start menu, check **Android Studio**.

d)(i) If **Android Studio** appears, opens it. On the Welcome screen, click **More Actions > SDK Manager**. Look at the top for **Android SDK Location**. Copy that path. It usually looks like this: ***C:\Users\YourName\AppData\Local\Android\Sdk***. Open that path in your File Explorer and look for the **platform-tools** folder inside it.

d)(ii) If nothing appears, download it at Google. Then, continue with step **d)(i)**.

e) Once you have that path (e.g., ***C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools***), add it to your **System Environment Variables** (under "Path"). Close and Reopen VS Code. Type **adb devices** in the terminal. If it returns a list (even an empty one), you have successfully linked it! Continue with step **b)**.

f) After completing steps **b)**, enter the 6-pins **Wi-Fi pairing code** and successfully paired.

g) Enter the command **adb connect [IP_ADDRESS]:[PORT]**, where [IP_ADDRESS]:[PORT] can be achieved at **Settings > Wireless debugging > IP address & Port**.

h) Change directory to the path where folders **lib** and **android** located (***C:\Users\Admin\Pictures\Camera Roll\flutter_application_1***).

i) Run the command **flutter run**.

j) Failed to run? Run the command **flutter pub get**. Update also the file **pubspec.yaml** (not the same one as at the **Flutter to Access Location** part).

```
dependencies:
  flutter:
```

```
    sdk: flutter
  geolocator: ^13.0.1  # Make sure this line exists
  camera: ^0.11.0+1
  http: ^1.2.1
```

k) In your terminal, type: **flutter clean**. Then type: **flutter pub get**. Then try to run again: **flutter run**.

l) Right now, if see this message: **Please enable Developer Mode in your system settings. Run start ms-settings:developer to open settings.**, run this at CMD: ***start ms-settings:developers***. Look for the toggle that says "**Developer Mode**" (Install apps from any source, including loose files) and switch it to **On**. Windows will ask if you're sure. Click **Yes**. Continue with step **i)**.

How to Run?

1. At your terminal, change directory to where your **server.js** file is located at.

2. Run the command **node server.js**.

***Note: You must install Node.js first.*

3. At your another terminal, change directory to where **main.dart** file is located at.

4. Run the command **flutter run**.

***Note: **main.dart** is your UI.*